OS Lab Assignment 7 — Threads, Semaphores, Deadlock, Priority Scheduling in xv6

Building and Running

```
make qemu-nox-2cpu       # same as: make qemu-nox            (Q1–Q3 and the 2-CPU Q4 run)
make qemu-nox-1cpu       # same as: make qemu-nox CPUS=1     (main Q4 run)
```

Inside xv6, run `threadtest`, `synctest`, `deadlock` and `priotest`. Run `threadtest` several times, because the size of the race changes from run to run, and use one CPU for `priotest` so that the order depends on priority alone.

Folder	Contents
`kernel/`	`proc.c`, `proc.h`, `sysproc.c`, `syscall.c`, `syscall.h`, `defs.h`, `param.h`, `trap.c`
`user/`	`threadtest.c`, `synctest.c`, `deadlock.c`, `priotest.c`, `user.h`, `usys.S`
`Makefile`	the four programs in `UPROGS`, the `-smp N,sockets=N` fix, no `-Werror`, `qemu-nox-1cpu` / `qemu-nox-2cpu` shortcuts
`output_logs/`	`full_session_2cpu.txt` (one continuous boot), `q4_priotest_1cpu.txt` (separate 1-CPU boot) and per-question logs. The Q1–Q3 logs and the 2-CPU Q4 log are excerpts of the full session, so their pids continue from one test to the next.
`figures/`	`q4_scheduling.png` (Figure 1)

System calls added: `clone` (26), `join` (27), `set_priority` (28), `get_priority` (29). `sem_init` (23) and `sem_wait` (24) were rewritten, and `sem_signal` (25) replaces A5's `sem_post`. `shm_get` (22) is also still in the table; it is not part of this assignment.

Q1 — Kernel-supported threads (`clone` / `join`)

Design and data structures. `struct proc` gets one new field, `int isthread`. `clone(fcn, arg, stack)` is built like `fork()`: `allocproc()` supplies a new PCB and kernel stack, but `np->pgdir = curproc->pgdir`, so the address space is **shared** rather than copied. The new thread inherits `sz`, the trap frame, the open files, the cwd and the parent's priority. On the caller's stack page the kernel pushes a fake return address (`0xffffffff`) and then `arg`, sets `tf->esp` to that frame and `tf->eip = fcn`, and sets `eax = 0`. `sys_clone` validates the stack with `argptr(2, &stack, PGSIZE)`, so a stack buffer that is not entirely inside the process image is rejected.

`join()` follows the structure of `wait()` but only looks at children with `isthread == 1`. It frees the kernel stack and **does not** call `freevm`, because the page directory still belongs to the creating process.

Changes to `wait` / `exit` / `fork`:

- `wait()` skips thread children, and calls `freevm()` only when `!p->isthread`. `init` is the exception: it also reaps threads that were orphaned, again without `freevm`.
- `exit()` wakes the parent (so `join()` returns) and, when it hands children to `init`, sets `killed` on any child that is a thread.
- `fork()` needs no change. Called from a thread it copies the shared page directory with `copyuvm`, so the child gets a private copy.

Result before synchronization, 3 threads × 100,000 increments, 2 CPUs:

Run	Final counter	Lost updates
1	271,794	28,206
2	211,379	88,621
3	223,534	76,466

The counter is a single global, and the parent sees the sum of the threads' updates in it, so the threads really do share one address space. Updates are lost because `counter++` is a separate load, add and store, and two threads on different CPUs can interleave between those steps.

Q2 — Counting semaphores

Design and data structures. In `proc.c`:

```
struct semaphore { int value; int wakeups; struct spinlock lk; };
struct { struct spinlock lock; struct semaphore sem[NSEM]; } semtable;   /* NSEM = 10 */
```

- `sem_init(id, v)` takes `semtable.lock`, then the semaphore's own `lk`, and sets `value = v`, `wakeups = 0`.
- `sem_wait` takes `lk` and decrements `value`. If the result is negative it sleeps with `sleep(s, &s->lk)` until `wakeups > 0`, then consumes one wakeup.
- `sem_signal` takes `lk` and increments `value`. If the result is `<= 0` there is a sleeper, so it adds one wakeup and calls `wakeup(s)`.
- A negative `value` therefore equals the number of blocked processes. The `wakeups` counter lets exactly one sleeper through per signal, although xv6's `wakeup()` wakes every process on the channel.
- **Locking.** `semtable.lock` protects the table while a semaphore is being (re)initialized. Waiting and signalling take only the per-semaphore lock, so unrelated semaphores never contend. The lock order is always `sem lock → ptable.lock` (the second is taken inside `sleep` and `wakeup`), and nothing takes them in the opposite order, so no cycle is possible.

Before/after comparison of the race condition:

Program	Synchronization	Final counter
`threadtest`	none	211,379 – 271,794 (wrong, differs per run)
`synctest` part 1	`sem_wait(0)` / `sem_signal(0)` around `counter++`	**300,000** (exact)

Producer–Consumer (`synctest` part 2). One producer and one consumer share a 5-slot circular buffer, with `empty = 5`, `full = 0` and `mutex = 1`. Twenty items were produced and consumed in FIFO order, and the printed buffer count is consistent with every produced/consumed line (`count=0, overflow=no, underflow=no` at the end). `empty` is what stops the producer at 5 items and `full` is what stops the consumer at 0. In the logged run the consumer kept up, so the count peaked at 2 and the producer never had to wait.

Q3 — Deadlock detection and the Banker's algorithm

Design. `deadlock.c` keeps static arrays `avail[m]`, `alloc[n][m]`, `request[n][m]`, `max[n][m]` and `need[n][m]`.

- `detect_deadlock(int *list)` sets `Work = Available`, and marks a process finished at the start if it holds nothing. It then repeatedly finds an unfinished process with `Request ≤ Work`, adds its allocation back to `Work` and prints the step. Whatever remains unfinished is deadlocked; the function returns how many there are and fills `list` with their indices.
- `is_safe_state()` is the Banker's safety check on `Need = Max − Allocation`, printing the safe sequence or `UNSAFE`.
- `request_resources(p, req)` checks `req ≤ Need` and `req ≤ Available`, allocates tentatively, calls `is_safe_state()` and rolls back if the result is unsafe.
- Detection looks at what processes are *requesting now* and finds a deadlock that already exists. Avoidance looks at what they *may* request (`Max`) and refuses requests that could lead to one.
- **Real deadlock.** The parent creates two pipes and forks A, then B. A keeps Pipe1's write end and blocks reading Pipe2; B keeps Pipe2's write end and blocks

reading Pipe1. Each child reports over a status pipe before blocking. After 100 ticks with no progress, the parent builds the matrices (each pipe end is a single-instance resource), runs detection and then kills both children.

Results. Tests 1–3 use the standard textbook matrices, so the expected answers are known in advance.

Test	Scenario	Expected	Program output
1	5×3 detection matrix	no deadlock	`no deadlock` (order P0, P2, P3, P4, P1)
2	same, P2 requests one more C	deadlock	`DEADLOCK among P1 P2 P3 P4`
3	Banker's, Available = (3,3,2)	safe	`SAFE <P1 P3 P4 P0 P2>`; P1's request (1,0,2) **granted**
4	after test 3, P0 requests (0,2,0)	unsafe	`UNSAFE` (0 of 5 can finish); request **denied**, rolled back
5	real processes A and B, two pipes	circular wait	`DEADLOCK (circular wait) among A B`

Available, Allocation and Request/Max/Need are printed at every step (`q3_deadlock.txt`).

Q4 — Priority-based preemptive scheduler with aging

Design. `struct proc` gains `priority` (1 = highest, 20 = lowest; `allocproc` sets 10) and `waitticks`.

- `scheduler()` scans the whole table starting just after the last slot it chose (`last`, a static shared by all CPUs and protected by `ptable.lock`) and runs the RUNNABLE process with the smallest priority number. Because a later process only wins with a strictly smaller number, equals are served in round-robin order.
- Preemption comes from xv6's timer-driven `yield()`: every tick re-runs the scheduler, so a higher-priority process that becomes runnable takes over at the next tick.
- Aging: `trap.c` calls `ageprocs()` from the CPU-0 timer interrupt, the same place `ticks` is incremented, so a waiting process ages once per global tick however many CPUs there are. It increments `waitticks` for every RUNNABLE process, and once a process exceeds `AGE_TICKS = 30` (so at least 31 ticks) its priority number drops by 1 (never below 1) and the counter resets. Being scheduled also resets the counter. Aging is permanent; the original priority is not restored.
- `set_priority(pid, p)` rejects `p` outside 1–20. `get_priority(pid)` returns −1 for an unknown pid.

`priotest` forks four identical CPU-bound children with priorities 2, 6, 12 and 20. The parent runs at priority 1 so forking is never delayed, and a monitor at priority 1 polls `get_priority()` for the priority-20 child and prints each aging step.

Completion time vs. priority (`CPUS=1`, `q4_priotest_1cpu.txt`):

Rank	Start priority	Completion (ticks)	Final priority
1	2	147	2
2	6	293	2
3	12	439	3
4	20	581	6

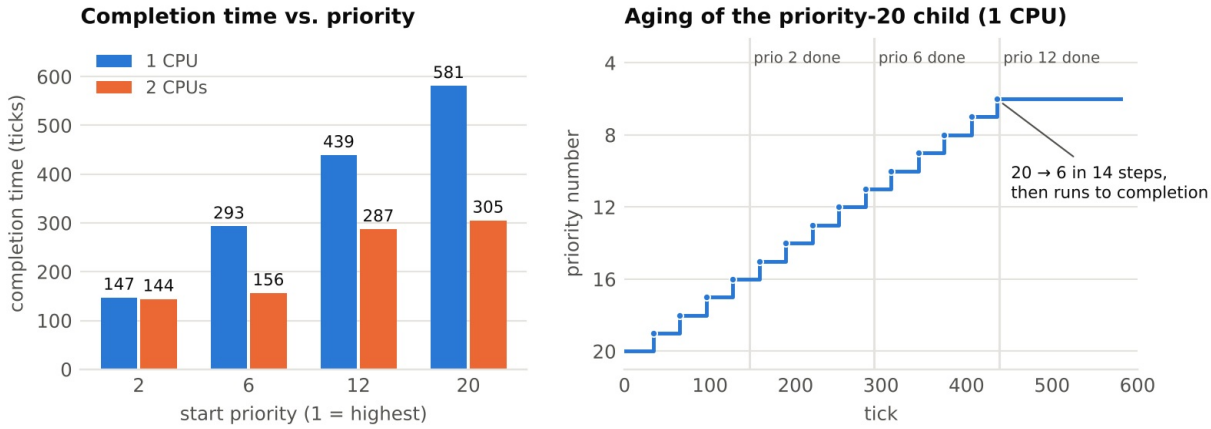


Figure 1. Left: completion time against start priority for both runs. Right: the priority number of the priority-20 child over time on one CPU (up = higher priority).

Completion order matches priority order exactly. The priority-20 child moved up one level roughly every 31 ticks (20 → 19 at tick 34, …, 7 → 6 at tick 436), then got the CPU and finished, so it did not starve. The gaps printed in the log vary between 30 and 34 because the monitor samples the clock when it polls. The completion ticks in the table are one higher than the matching `FINISHED` lines in the log (146 / 292 / 438 / 580). This is a one-tick reading difference and does not change the order.

On 2 CPUs (`q4_priotest_2cpu.txt`) the order is still 2, 6, 12, 20 (144 / 156 / 287 / 305 ticks), but the two highest-priority jobs run at the same time and the last two take the CPUs as they free up, so the one-CPU run is the cleaner test of priority.

Problems faced and how they were resolved

1. **No race visible, and only `cpu0` booting.** QEMU 10 turns `-smp 2` into one socket with two cores, and the MP table that xv6 parses then lists one CPU. I found this by printing `ncpu` at boot, which gave 1. The fix was `-smp $(CPUS),sockets=$(CPUS)` in the Makefile, after which `cpu1: starting 1` appears.
2. **Threads finished before the next one started**, so nothing overlapped. I added a `volatile int go` start barrier that the parent sets after all threads have been created.
3. **Build failure with GCC 15.** `-Werror` turned the `-Warray-bounds` warnings in stock `mp.c` into errors, so I removed `-Werror`, as in the earlier assignments.
4. **Lost wakeups and the thundering herd.** `wakeup()` wakes every sleeper on a channel, so the semaphore keeps a `wakeups` count and lets exactly one sleeper through per signal.
5. **`struct spinlock` inside `proc.h`** broke files that include `proc.h` before `spinlock.h`. I moved `struct semaphore` into `proc.c`, its only user.
6. **`priotest` hung after the first two jobs.** Each child looped on `sleep(1)` until its priority equalled the value assigned to it, but aging changed the priority while it waited, so the loop never ended. I removed the handshake: the priority-1 parent now sets each child's priority right after `fork()`.
7. **Interleaved output** from the two deadlock children on separate CPUs. The parent now forks B only after A has reported.

Known limitations

- **Exiting with live threads.** If a process exits while its threads are still running, its page directory is freed when the parent reaps it, even though the threads are marked `killed`. All test programs `join()` every thread first.
- **Per-thread `sz`.** Each PCB keeps its own copy of `sz`, so threads must not call `sbrk` or `malloc`. The test programs allocate every thread stack (at least `PGSIZE` bytes) before calling `clone()`.
- **Re-initializing a semaphore** with `sem_init` while processes are sleeping on it is not supported.